

Section 2 / 9

[Go to Questions](#) ↓

Information Disclosure

Exploiting any service requires thorough enumeration and reconnaissance to identify all possible attack vectors. As attackers, we aim to obtain as much information about a service as possible.

Identifying the GraphQL Engine

After logging in to the sample web application and investigating all functionality, we can observe multiple requests to the `/graphql` endpoints that contain GraphQL queries:

Request	Response
<pre>1 POST /graphql HTTP/1.1 2 Host: 172.17.0.2 3 Content-Length: 67 4 Content-Type: application/json 5 Cookie: session= eyJyb2x1IjoiaXNlciIsInVzZXIiOiJodGItc3RkbmQifQ.ZqKJBg.eNKX254dZAS 4lrVgkX9E2VTuSiM 6 7 { "query": "{posts { uuid title body category author { username } }}" }</pre>	<pre>1 HTTP/1.1 200 OK 2 Server: Werkzeug/3.0.3 Python/3.11.2 3 Date: Thu, 25 Jul 2024 17:19:56 GMT 4 Content-Type: application/json 5 Content-Length: 2974 6 Connection: close 7 8 { "data": { "posts": [{ "uuid": "1", "title": "Lorem ipsum 1", "body": "Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolo</pre>

Thus, we can definitely say that the web application implements GraphQL. As a first step, we will identify the GraphQL engine used by the web application using the tool `graphw00f`. Graphw00f will send various GraphQL queries, including malformed queries, and can determine the GraphQL engine by observing the backend's behavior and error messages in response to these queries.

After cloning the git repository, we can run the tool using the `main.py` Python script. We will run the tool in fingerprint (`-f`) and detect mode (`-d`). We can provide the web application's base URL to let graphwoof attempt to find the GraphQL endpoint by itself:

shellsession

```
m3m0rydmp@htb[/htb]$ python3 main.py -d -f -t http://172.17.0.2
```

```

+-----+
|  graphw00f  |
+-----+
***          ***
**          **
**          **
**          **

+-----+      +-----+
|  Node X  |      |  Node Y  |
+-----+      +-----+
***          ***
**          **
**          **
**          **

+-----+
|  Node Z  |
+-----+

```

graphw00f - v1.1.17

The fingerprinting tool for GraphQL

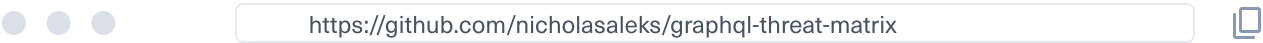
Dolev Farhi <dolev@lethalbit.com>

```

[*] Checking http://172.17.0.2/
[*] Checking http://172.17.0.2/graphql
[!] Found GraphQL at http://172.17.0.2/graphql
[*] Attempting to fingerprint...
[*] Discovered GraphQL Engine: (Graphene)
[!] Attack Surface Matrix: https://github.com/nicholasaleks/graphql-
threat-matrix/blob/master/implementations/graphene.md
[!] Technologies: Python
[!] Homepage: https://graphene-python.org
[*] Completed.

```

As we can see, the graphwoof identified the GraphQL engine [Graphene](#) . Additionally, it provides us with the corresponding detailed page in the [GraphQL-Threat-Matrix](#), which provides more in-depth information about the identified GraphQL engine:



Security Considerations

graphene provides the following features which should be taken into consideration:

Field Suggestions	Query Depth Limit	Query Cost Analysis	Automatic Persisted Queries	Introspection	Debug Mode	Batch Requests
✔ Enabled by Default	✗ No Support	✗ No Support	✗ No Support	✔ Enabled by Default	✗ No Support	⚠ Disabled by Default

Lastly, by accessing the `/graphql` endpoint in a web browser directly, we can see that the web application runs a [graphiql](#) interface. This enables us to provide GraphQL queries directly, which is a lot more convenient than running the queries through Burp, as we do not need to worry about breaking the JSON syntax.

Introspection

[Introspection](#) is a GraphQL feature that enables users to query the GraphQL API about the structure of the backend system. As such, users can use introspection queries to obtain all queries supported by the API schema. These introspection queries query the `__schema` field.

For instance, we can identify all GraphQL types supported by the backend using the following query:

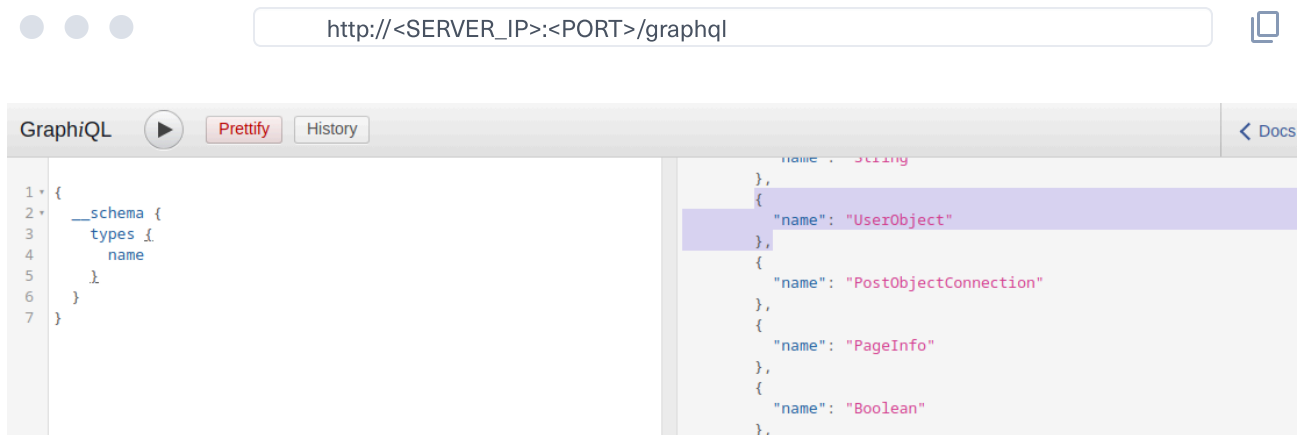
```
graphql
{
  __schema {
    types {
      name
```

```

    }
  }
}

```

The results contain basic default types, such as `Int` or `Boolean` , but also all custom types, such as `UserObject` :



Now that we know a type, we can follow up and obtain the name of all of the type's fields with the following introspection query:

```

graphql
{
  __type(name: "UserObject") {
    name
    fields {
      name
      type {
        name
        kind
      }
    }
  }
}

```

In the result, we can see details we would expect from a user object, such as `username` and `password` , as well as their data types:



Furthermore, we can obtain all the queries supported by the backend using this query:

```

graphql
{
  __schema {
    queryType {
      fields {
        name
        description
      }
    }
  }
}

```

Knowing all supported queries helps us identify potential attack vectors that we can use to obtain sensitive information. Lastly, we can use the following "general" introspection query that dumps all information about types, fields, and queries supported by the backend:

```

graphql
query IntrospectionQuery {
  __schema {
    queryType { name }
    mutationType { name }
    subscriptionType { name }
  }
}

```

```
types {
  ...FullType
}
directives {
  name
  description

  locations
  args {
    ...InputValue
  }
}
}
```

```
fragment FullType on __Type {
  kind
  name
  description

  fields(includeDeprecated: true) {
    name
    description
    args {
      ...InputValue
    }
    type {
      ...TypeRef
    }
    isDeprecated
    deprecationReason
  }
  inputFields {
    ...InputValue
  }
  interfaces {
    ...TypeRef
  }
  enumValues(includeDeprecated: true) {
    name
```

```
    description
    isDeprecated
    deprecationReason
  }
  possibleTypes {
    ...TypeRef
  }
}

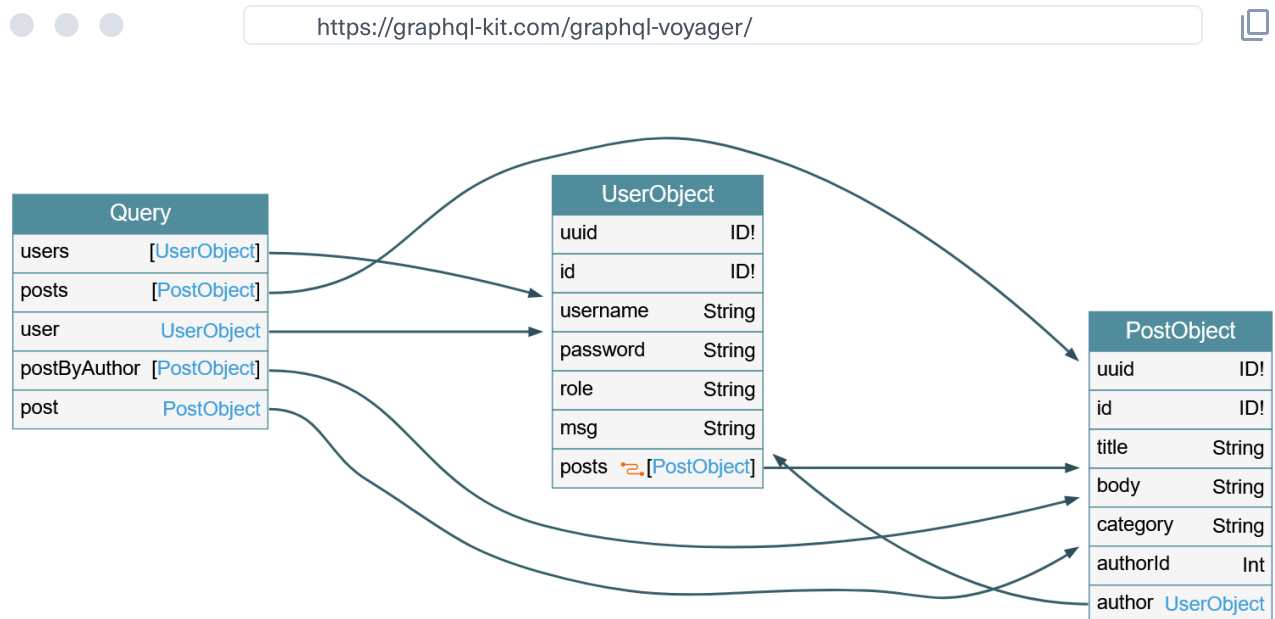
fragment InputValue on __InputValue {
  name
  description
  type { ...TypeRef }
  defaultValue
}

fragment TypeRef on __Type {
  kind
  name
  ofType {
    kind
    name
    ofType {
      kind
      name
      ofType {
        kind
        name
        ofType {
          kind
          name
          ofType {
            kind
            name
            ofType {
              kind
              name
            }
          }
        }
      }
    }
  }
}
```

```
}  
}  
}  
}  
}  
}  
}  
}
```

The result of this query is quite large and complex. However, we can visualize the schema using the tool [GraphQL-Voyager](#). For this module, we will use the [GraphQL Demo](#). However, in a real engagement, we should follow the GitHub instructions to host the tool ourselves, ensuring that no sensitive information leaves our system.

In the demo, we can click [CHANGE SCHEMA](#) and select [INTROSPECTION](#). After pasting the result of the above introspection query in the text field and clicking on [DISPLAY](#), the backend's GraphQL schema is visualized for us. We can explore all supported queries, types, and fields:



Connect to HTB

Target(s)

Spawn the target system to get IPs and answer questions

 **Spawn the target system**



Enable step-by-step solutions  **PRO**

Question 1

 +2

 +40



After executing an introspection query, what is the flag you can exfiltrate?

Authenticate to with user "**htb-stdnt**" and password "**AcademyStudent!**"

Write your answer

 **Submit**



2 / 9 Sections



Mark Complete & Next